

Agenda

1. Syntax and core language
2. Iterators and how that is working
3. Objects and protocols
4. Tools and dataclasses

History

Python 2.x was very popular. Guido said: The time has come to improve Python.

He called this "Python 3000." The goal was to rewrite the language to be more consistent and more modern.

Python 3 came out, and was immediately hated. Why? Because it was incompatible.

Incompatibility #1 -- `print`

`print` is now a function!

You used to be able to say `print 'hello'`

```
In [1]: print 'hello'
```

```
Cell In[1], line 1
    print 'hello'
    ^
SyntaxError: Missing parentheses in call to 'print'. Did you mean print (...)?
```

```
In [2]: help(print)
```

Help on built-in function print in module builtins:

```
print(*args, sep=' ', end='\n', file=None, flush=False)
```

Prints the values to a stream, or to sys.stdout by default.

sep

string inserted between values, default a space.

end

string appended after the last value, default a newline.

file

a file-like object (stream); defaults to the current sys.stdout.

flush

whether to forcibly flush the stream.

In [3]: **True**

Out[3]: True

In [4]: **False**

Out[4]: False

In [5]: *# True, False = False, True*

In [6]: *# ints and bigints -- no more!*
integers can be as big as you want

```
x = 100_00_000  
type(x)
```

Out[6]: int

In [9]: x = x ** 50

In [10]: x

```
In [11]: type(x)
```

```
In [12]: x = x ** 50
```

```
In [13]: type(x)
```

Now things have reversed: Strings only work with Unicode, and if you want bytes, you need a "bytestring"

```
In [15]: s = 'abcde'
len(s)
```

```
Out[15]: 5
```

```
In [16]: s = 'אבגדה'
len(s)
```

```
Out[16]: 5
```

```
In [17]: s[0]
```

```
Out[17]: 'א'
```

```
In [18]: ord(s[0]) # this gives me the Unicode number, not ASCII
```

```
Out[18]: 1488
```

```
In [19]: import unicodedata
```

```
In [21]: unicodedata.name(s[0])
```

```
Out[21]: 'HEBREW LETTER ALEF'
```

```
In [22]: l/ # L/ukas -- Wukash
```

```
Cell In[22], line 1
    l/ # L/ukas -- Wukash
      ^
SyntaxError: invalid syntax
```

```
In [23]: # what if I want to read in a binary file?
# Or what if I want to work with bytes?
# I need to use a bytestring

s
```

```
Out[23]: 'אבגדה'
```

```
In [25]: b = s.encode() # this returns a bytestring
b
```

```
Out[25]: b'\xd7\x90\xd7\x91\xd7\x92\xd7\x93\xd7\x94'
```

```
In [26]: b[0]
```

```
Out[26]: 215
```

If you read a binary file into Python, you must read it with `'rb'` and you will not get a string, but rather a bytestring

```
In [27]: b[1:]
```

```
Out[27]: b'\x90\xd7\x91\xd7\x92\xd7\x93\xd7\x94'
```

```
In [28]: b[1:].decode() # let's turn this into a string
```

```
-----  
UnicodeDecodeError                                Traceback (most recent call last)  
Cell In[28], line 1  
----> 1 b[1:].decode() # let's turn this into a string  
  
UnicodeDecodeError: 'utf-8' codec can't decode byte 0x90 in position 0: invalid start byte
```

Other big string news: f-strings!

These were introduced in Python 3.7. They replace two previous string interpolation systems:

- We could use `%` as an operator on strings -- this still exists, and is used in the `logging` module
- We can also use `str.format`, a method that lets us interpolate

```
In [29]: x = 10  
y = 20  
z = [10, 20, 30]  
  
print(f'{x} + {y} = {x+y}')
```

```
10 + 20 = 30
```

```
In [30]: print(f'{z}')
```

```
[10, 20, 30]
```

```
In [31]: f'{x=}' # this means: print the expression and its value
```

```
Out[31]: 'x=10'
```

```
In [33]: f'{x = }'
```

```
Out[33]: 'x = 10'
```

```
In [34]: f'{len(z)=}'
```

```
Out[34]: 'len(z)=3'
```

```
In [36]: s = 'abcdefg'
```

```
print(f'{s!r}') # this returns the repr of the object
```

```
'abcdefg'
```

```
In [37]: print(f'{z!r}')
```

```
[10, 20, 30]
```

```
In [38]: # repr == the object's internal representation, as shown to programmers in J
```

```
In [39]: d = {'a':1000, 'bcdef':2, 'ghi':300}

for key, value in d.items():
    print(f'{key}: {value}')
```

```
a: 1000
bcdef: 2
ghi: 300
```

Inside of the `{}` for the f-string, you can put a `:`. After the `:` comes a format specifier. Every type has its own format specifiers. For strings, you can indicate the size of the field you want to use.

```
In [46]: d = {'a':1000, 'bcdef':2, 'ghi':300}

for key, value in d.items():
    print(f'{key:.<10}{value:.>10}')
```

```
a.....1000
bcdef.....2
ghi.....300
```

fstring help: <https://fstring.help/>

```
In [52]: n = 12345.67890

print(f'{n:.<3f}')
```

```
12345.679
```

```
In [53]: print(f'{n:3%}')
```

```
1234567.890000%
```

Unpacking (aka tuple unpacking)

```
In [54]: mylist = [10, 20, 30]

x = mylist

x
```

```
Out[54]: [10, 20, 30]
```

```
In [55]: x,y,z = mylist    # 3 variables on the left, 3 items on the right

x
```

```
Out[55]: 10
```

```
In [56]: y
```

```
Out[56]: 20
```

```
In [57]: z
```

```
Out[57]: 30
```

```
In [58]: # Python 3 improved things with splat
```

```
In [59]: mylist = [10, 20, 30, 40, 50, 60, 70, 80]
```

```
x, y, z = mylist
```

```
-----  
ValueError                                Traceback (most recent call last)  
Cell In[59], line 3  
      1 mylist = [10, 20, 30, 40, 50, 60, 70, 80]  
      2  
----> 3 x, y, z = mylist  
ValueError: too many values to unpack (expected 3, got 8)
```

```
In [60]: x, y, *z = mylist
```

```
In [61]: x
```

```
Out[61]: 10
```

```
In [62]: y
```

```
Out[62]: 20
```

```
In [63]: z
```

```
Out[63]: [30, 40, 50, 60, 70, 80]
```

```
In [64]: x, *y, z = mylist
```

```
In [65]: y
```

```
Out[65]: [20, 30, 40, 50, 60, 70]
```

```
In [66]: x, *trash, z = mylist
```

```
In [67]: x
```

```
Out[67]: 10
```

```
In [68]: z
```

```
Out[68]: 80
```

```
In [69]: x, *_ , z = mylist # I don't do this, but many people do
```

```
In [70]: type(x)
```

```
Out[70]: int
```

```
In [71]: mylist = 'this is a bunch of words'.split()
```

```
In [72]: x, *_ , z = mylist
```

```
In [73]: x
```

```
Out[73]: 'this'
```

```
In [74]: z
```

```
Out[74]: 'words'
```

```
In [75]: _
```

```
Out[75]: ['is', 'a', 'bunch', 'of']
```

```
In [76]: b
```

```
Out[76]: b'\xd7\x90\xd7\x91\xd7\x92\xd7\x93\xd7\x94'
```

```
In [78]: type(b[0])
```

```
Out[78]: int
```

Exercise: Format receipt

Given a list of 3-element tuples (product name, unit price, and quantity), print a receipt with names padded to a fixed width, prices to two decimals, and a computed total per row.

Write a function, `print_receipt`, that takes a list of such 3-element tuples and prints on the screen.

<https://practice.lernerpython.com/classroom/bfb596ff6a/ex-1>

```
In [91]: def print_receipt(items):  
        # item is a bunch of tuples  
        # each tuples will be name, price, quantity  
  
        for one_item in items:  
            name, price, quantity = one_item  
            print(f'{name:10}\t{price:^6.2f}\t${quantity * price:.2f}')
```



```

items = [
    ('Widget', 9.99, 3),
    ('Gadget', 24.50, 1),
    ('Doohickey', 4.75, 12),
]

print_receipt(items)

```

Widget	9.99	\$29.97
Gadget	24.50	\$24.50
Doohickey	4.75	\$57.00

Structural pattern matching

For *YEARS*, people new to Python complained that it didn't have a switch-case statement. And then, Python added match-case, and said that you can use this instead of `if`.

Except that it's not the same:

- `switch-case` in C is for `if - else` decision-making
- `match - case` in Python is for breaking apart data structures without having to work hard

```

In [92]: name = 'Reuven'

match name:
    case 'Reuven':
        print('Hi, boss!')
    case 'someone else':
        print('Hi, someone else')
    case _:
        print('This is weird')

```

Hi, boss!

Traditionally, the final `case` is put into a variable. That means: It'll match anything, anything at all. The `_` variable is used for throwaway data.

```

In [97]: name = 'Reasdfsadfasdfdasdfdadsuven'

match name:
    case 'Reuven':
        print('Hi, boss!')
    case 'someone else':
        print('Hi, someone else')
    case other_name: # the variable here is assigned the value in the "match"
        print('This is weird')

```

This is weird

In [98]: other_name

Out[98]: 'Reasdfsadfasdfdasdfdadsuven'

In [101]... *# structural pattern matching is really meant for picking apart values from*

```
mylist = [10, 20]

match mylist:
    case [x]:
        print(f'{x=}')
    case [x, y]:
        print(f'{x=}, {y=}')
    case [x, y, z]:
        print(f'{x=}, {y=}, {z=}')
    case other:
        print(f'{other=}')
```

x=10, y=20

In [105]... *# we can use our unpacking *z stuff here, too:*

```
mylist = [10, 20, 30, 40, 50, 60]

match mylist:
    case [x]:
        print(f'{x=}')
    case [x, y]:
        print(f'{x=}, {y=}')
    case [x, y, *z]:
        print(f'{x=}, {y=}, {z=}')
    case other:
        print(f'{other=}')
```

x=10, y=20, z=[30, 40, 50, 60]

In [107]... *# we can match dicts, too:*

```
d = {'a':10, 'b':20, 'c':30}

match d:
    case {'a':10, 'b':20}:
        print('Partial match')
    case _:
        print('Default')
```

Partial match

In [112]... name = 'asdfsafdsdafsafsafsafsafas'

```
match name:
    case x if len(x) < 5:
        print(f'Short name, {name}!')
    case x if len(x) > 10:
        print(f'Long name, {name}!')
```

```
case _:  
    print(f'Medium name, {name}!')
```

Long name, asdfsafdsdafsafsafsafsafas!

Next up

1. Modern dicts
2. Comprehensions
3. Iteration

Resume at the top of the hour

Modern dicts

For many years, one of the most important things to tell people about dicts was that they didn't preserve their order. If you created `{'a':10, 'b':20, 'c':30}`, you didn't know if iterating would give you `a`, `b`, `c`, or maybe `b`, `c`, and `a`.

Then, in Python 3.6 and 3.7, they changed this entirely: In 3.6, they changed the implementation of dicts, and in 3.7 they said that a nice side feature (namely, the ordering) was guaranteed.

What did they change?

A dict is a key-value store, a hash table. This means that if you store `d['a'] = 10`, Python runs the `hash('a')`, gets a number, and (in old versions of Python) would use that number to know where to store your value. Iterating over a dict meant going through each of the keys, invoking `hash` on each key, and retrieving each value.

Not only did this not guarantee the order, but it also meant that the dict needed a *lot* of space in memory. If we didn't use lots of memory, then we would end up with many collisions, which is a problem in dicts.

In 3.6, they changed the implementation to use 2 data structures:

1. a table with three columns that grows over time. This is where the key and value are stored, sequentially. The third column is the index in the table, starting with 0. There's one row in this table for each key-value pair in the dict.
2. an array (at the C level), which is where the hash function is really invoked. This is much smaller than the original dict ever was, because it's just integers in C.

If you store `d['a'] = 10`, Python first puts a new row in the table with the index (automatically calculated) and then the key-value pair. Then Python calculates

`hash('a') % len(array)` , and gets an indication of where to put the index from the table.

So the array contains table-row indexes. Each lookup in the dict requires two lookups -- one in the array and one in the dict.

But we save a TON of memory!

Not only does this save memory, but we also have the table with our key-value pairs in chronological order!

What happens, though, when we remove a key-value pair?

Remember that all variables in Python are actually keys in a dict. Remember that every attribute (i.e., names after `.`) are really dicts. Remember that objects are really dicts.

```
In [113... d = {'a':10, 'b':20, 'c':30}

for key, value in d.items():
    print(f'{key}: {value}')
```

```
a: 10
b: 20
c: 30
```

```
In [114... d.pop('b')
d['b'] = 999

for key, value in d.items():
    print(f'{key}: {value}')
```

```
a: 10
c: 30
b: 999
```

What else has changed?

We have new dict operators!

```
In [116... d1 = {'a':10, 'b':20, 'c':30}
d2 = {'b':200, 'd':400}

# we could always say:

d1.update(d2) # this means: let d2 overwrite d1
d1
```

```
Out[116... {'a': 10, 'b': 200, 'c': 30, 'd': 400}
```

```
In [117... # in Python 3.9 , we have the | operator, which does the same, but non-destr
```

```
d1 = {'a':10, 'b':20, 'c':30}
d2 = {'b':200, 'd':400}

d1 | d2
```

```
Out[117...] {'a': 10, 'b': 200, 'c': 30, 'd': 400}
```

```
In [118...] d1
```

```
Out[118...] {'a': 10, 'b': 20, 'c': 30}
```

```
In [119...] d2
```

```
Out[119...] {'b': 200, 'd': 400}
```

```
In [120...] # you can also say
```

```
d1 |= d2    # now this is the same as dict.update
d1
```

```
Out[120...] {'a': 10, 'b': 200, 'c': 30, 'd': 400}
```

```
In [121...] d.keys()
```

```
Out[121...] dict_keys(['a', 'c', 'b'])
```

```
In [122...] d.values()
```

```
Out[122...] dict_values([10, 30, 999])
```

`dict_keys` and `dict_values` are set-like data structures. You can use many set operators on them, including `|` (union) and others.

```
In [123...] d1.keys() | d2.keys()
```

```
Out[123...] {'a', 'b', 'c', 'd'}
```

Comprehensions

Comprehensions are still a major part of Python -- and in fact, we probably use them even more!

- List comprehensions
- Dict comprehensions
- Set comprehensions
- Generator expressions

One big change in modern Python is that comprehensions *don't* leak their variables to the outside world!

```
In [124... x = 100

[x**2
 for x in range(5)]
```

```
Out[124... [0, 1, 4, 9, 16]
```

```
In [125... x
```

```
Out[125... 100
```

Normally, if we don't have a function, then we don't have a local scope. Does this mean that a comprehension is really a function in disguise?

The answer is: Yes, it used to be! For the first part of Python 3's life, a comprehension did create an anonymous, private function that gave it locally scoped variables.

That is no longer the case, but the variables are still locally scoped.

```
In [126... # this is not the case with regular for loops:
```

```
x = 100

for x in range(5):
    print(x**2)

print(x)
```

```
0
1
4
9
16
4
```

The great de-listing

Back in Python 2, we had two different `range` functions:

- `range(n)` , which returned a list of integers from 0 to `n-1`
- `xrange(n)` , which returned an iterable object that didn't contain all of the numbers, but could generate them in a `for` loop.

Why use `range` in such a world? Just use `xrange` -- for small things, no one cares, and for big things, everyone cares!

In Python 3, they got rid of `range` , and replaced it with `xrange` , which they renamed to `range` . Basically, our modern `range` is the old `xrange` .

```
In [127... r = range(100)
r[3]
```

Out[127... 3

The larger story is that Python is now being used for large-scale data. The notion that we'll get lists back from things is a bit scary -- if the list is too big, it'll blow up our computer's memory and/or crash our program.

Everything is moving in the direction of generators -- objects that are small and don't contain all of the values. Rather, they know how to compute the values, and will give us the next one with each iteration.

Just about everything that used to provide a list now provides a generator, meaning that we don't have to worry so much about memory when we ask for a lot of things.

Some examples: `enumerate`, `zip`, `itertools` -- these no longer provide lists, but instead provide iterators that we can use in a `for` loop.

```
In [128... s = 'abcd'

for index, one_character in enumerate(s):
    print(f'{index}: {one_character}')
```

```
0: a
1: b
2: c
3: d
```

```
In [129... enumerate(s)
```

Out[129... <enumerate at 0x109ef23e0>

```
In [130... s = 'abcd'
r = range(4)

zip(s, r) # get tuples back with (s[0], r[0]) and then (s[1], r[1]) until s
```

Out[130... <zip at 0x109f22200>

```
In [131... list(zip(s, r)) # potentially dangerous, if it's a lot of data
```

Out[131... [('a', 0), ('b', 1), ('c', 2), ('d', 3)]

```
In [132... s = 'abcd e'

for index, one_character in enumerate(s):
    print(f'{index}: {one_character}')
```

```
0: a
1: b
2: c
3: d
4:
5: e
```

In [134... *# we can create our own generators as functions, using yield to return + pause*

```
def my_enumerate(data):
    index = 0
    for one_item in data:
        return index, one_item
        index += 1

my_enumerate(s)
```

Out[134... (0, 'a')

In [135... **def** my_enumerate(data):
 index = 0
 for one_item **in** data:
 yield index, one_item *# this means: return these values and go to s*
 index += 1

my_enumerate(s) *# Python sees the yield in the function, and knows to ta*

Out[135... <generator object my_enumerate at 0x1099d10e0>

In [136... list(my_enumerate(s))

Out[136... [(0, 'a'), (1, 'b'), (2, 'c'), (3, 'd'), (4, ' '), (5, 'e')]

In [137... **for** index, one_character **in** my_enumerate(s):
 print(f'{index}: {one_character}')

```
0: a
1: b
2: c
3: d
4:
5: e
```

In [139... s.split() *# this returns a list of strings, by default using whitespace as a*

Out[139... ['abcd', 'e']

In [140... **for** index, one_word **in** my_enumerate(s.split()):
 print(f'{index}: {one_word}')

```
0: abcd
1: e
```

In [141... **def** my_enumerate(data):
 index = 0
 for one_item **in** data:


```
    return index, one_item
    index += 1
```

```
In [143... import dis # Python disassembler
```

```
In [144... dis.show_code(my_enumerate)
```

```
Name:          my_enumerate
Filename:      /var/folders/rr/0mnyyv811fs5vyp22gf4fxk00000gn/T/ipykerne
l_23927/2398052561.py
Argument count: 1
Positional-only arguments: 0
Kw-only arguments: 0
Number of locals: 3
Stack size:    3
Flags:         OPTIMIZED, NEWLOCALS
Constants:
  0: 0
  1: None
Variable names:
  0: data
  1: index
  2: one_item
```

```
In [145... dis.dis(my_enumerate)
```

1	RESUME	0
2	LOAD_SMALL_INT	0
	STORE_FAST	1 (index)
3	LOAD_FAST_BORROW	0 (data)
	GET_ITER	
	FOR_ITER	6 (to L1)
	STORE_FAST	2 (one_item)
4	LOAD_FAST_BORROW_LOAD_FAST_BORROW	18 (index, one_item)
	BUILD_TUPLE	2
	SWAP	2
	POP_TOP	
	RETURN_VALUE	
3 L1:	END_FOR	
	POP_ITER	
	LOAD_CONST	1 (None)
	RETURN_VALUE	

```
In [146... def my_enumerate(data):
    index = 0
    for one_item in data:
        yield index, one_item
        index += 1
```

```
In [147... dis.show_code(my_enumerate)
```

```

Name:          my_enumerate
Filename:      /var/folders/rr/0mnyyv811fs5vyp22gf4fxk00000gn/T/ipykerne
l_23927/146466865.py
Argument count: 1
Positional-only arguments: 0
Kw-only arguments: 0
Number of locals: 3
Stack size:    3
Flags:         OPTIMIZED, NEWLOCALS, GENERATOR
Constants:
    0: 0
    1: None
Variable names:
    0: data
    1: index
    2: one_item

```

```
In [149... bin(my_enumerate.__code__.co_flags)
```

```
Out[149... '0b100011'
```

```
In [150... dis.dis(my_enumerate)
```

```

1          RETURN_GENERATOR
          POP_TOP
      L1:    RESUME                    0

2          LOAD_SMALL_INT            0
          STORE_FAST                  1 (index)

3          LOAD_FAST_BORROW           0 (data)
          GET_ITER
      L2:    FOR_ITER                  17 (to L3)
          STORE_FAST                  2 (one_item)

4          LOAD_FAST_BORROW_LOAD_FAST_BORROW 18 (index, one_item)
          BUILD_TUPLE                  2
          YIELD_VALUE                  0
          RESUME                        5
          POP_TOP

5          LOAD_FAST_BORROW           1 (index)
          LOAD_SMALL_INT              1
          BINARY_OP                    13 (+=)
          STORE_FAST                  1 (index)
          JUMP_BACKWARD                19 (to L2)

3   L3:     END_FOR
          POP_ITER
          LOAD_CONST                    1 (None)
          RETURN_VALUE

--   L4:     CALL_INTRINSIC_1           3 (INTRINSIC_STOPITERATION_ERROR)
          RERAISE                      1

ExceptionTable:
  L1 to L4 -> L4 [0] lasti

```

```
In [152... g = my_enumerate(s) # grab the generator
```

```
In [154... g.gi_frame.f_lasti
```

```
Out[154... 2
```

```
In [155... next(g)
```

```
Out[155... (0, 'a')
```

```
In [156... g.gi_frame.f_lasti
```

```
Out[156... 26
```

```
In [157... g.gi_frame.f_locals
```

```
Out[157... {'data': 'abcd e', 'index': 0, 'one_item': 'a'}
```

My talk about generators and "nanoservices": <https://www.youtube.com/watch?v=tkoaeVS2zRQ>

```
In [158... def myfunc():
    mylist = [10, 20, 30]
    print(mylist[0])
```

```
In [159... dis.dis(myfunc)
```

1	RESUME	0
2	BUILD_LIST	0
	LOAD_CONST	2 ((10, 20, 30))
	LIST_EXTEND	1
	STORE_FAST	0 (mylist)
3	LOAD_GLOBAL	1 (print + NULL)
	LOAD_FAST_BORROW	0 (mylist)
	LOAD_SMALL_INT	0
	BINARY_OP	26 ([])
	CALL	1
	POP_TOP	
	LOAD_CONST	1 (None)
	RETURN_VALUE	

```
In [160... myfunc.__code__.co_consts
```

```
Out[160... (10, None, (10, 20, 30))
```

```
In [161... def myfunc():
    mylist = [10, 20, 30]
    return mylist[0]
```

```
In [162... dis.dis(myfunc)
```

1	RESUME	0
2	BUILD_LIST	0
	LOAD_CONST	1 ((10, 20, 30))
	LIST_EXTEND	1
	STORE_FAST	0 (mylist)
3	LOAD_FAST_BORROW	0 (mylist)
	LOAD_SMALL_INT	0
	BINARY_OP	26 ([])
	RETURN_VALUE	

Iterator protocol

Python is using generators more and more, and they are objects that know how to use the iterator protocol. This protocol is not new, but it's just everywhere now:

1. An iterable object should respond to the `iter` builtin function, returning its iterator.
 - If it's not iterable, then it raises an exception
2. An iterator should respond to the `next` builtin function, returning the next value it has to offer.
 - If it's at the end, then it raises the `StopIteration` exception

```
In [163... e = my_enumerate(s)
```

```
In [164... iter(e)
```

```
Out[164... <generator object my_enumerate at 0x10a0025e0>
```

```
In [165... iter(e)
```

```
Out[165... <generator object my_enumerate at 0x10a0025e0>
```

```
In [166... next(e) # give me your next value
```

```
Out[166... (0, 'a')
```

```
In [167... next(e)
```

```
Out[167... (1, 'b')
```

```
In [168... next(e)
```

```
Out[168... (2, 'c')
```

```
In [169... next(e)
```

```
Out[169... (3, 'd')
```

```
In [170...] next(e)
```

```
Out[170...] (4, ' ')
```

```
In [171...] next(e)
```

```
Out[171...] (5, 'e')
```

```
In [172...] next(e)
```

```
-----  
StopIteration                                Traceback (most recent call last)  
Cell In[172], line 1  
----> 1 next(e)  
  
StopIteration:
```

```
In [173...] iter(5)
```

```
-----  
TypeError                                Traceback (most recent call last)  
Cell In[173], line 1  
----> 1 iter(5)  
  
TypeError: 'int' object is not iterable
```

Behind the scenes, what does it mean for an object to know how to respond to `iter` or `next` ?

The answer: It implements two methods.

- If an object implements the `__iter__` method, then it is considered iterable. This method should return an iterator. (It could return the object itself, if the object implements `__next__`.)
- If an object implements the `__next__` method, then it is an iterator. This method will be invoked repeatedly by `next`, until we get a `StopIteration` exception, which stops the loop.

```
In [177...] # to make any class iterable, just implement those methods
```

```
class MyData:  
    def __init__(self, data):  
        self.data = data  
        self.index = 0  
  
    def __iter__(self): # I'm my own iterator! -- this method is invoked on  
        return self  
  
    def __next__(self):  
        if self.index >= len(self.data): # if we're at the end, indicate we  
            raise StopIteration
```

```

        value = self.data[self.index]
        self.index += 1
        return value

m = MyData('abcd')

for one_item in m:
    print(one_item)

```

a
b
c
d

In [179... *# we can instead have __iter__ return a generator, via a generator function*

```

class MyData:
    def __init__(self, data):
        self.data = data

    def __iter__(self):
        for one_item in self.data:
            yield one_item

m = MyData('abcd')

for one_item in m:
    print(one_item)

```

a
b
c
d

In [180... iter(m)

Out[180... <generator object MyData.__iter__ at 0x10a06c1e0>

In [183... iter(m).__next__()

Out[183... 'a'

In [185... from itertools import chain

In [186...

```

s = 'abcd'
mylist = [10, 20, 30]
t = (100, 200, 300)

for one_item in chain(s, mylist, t):
    print(one_item)

```

```
a
b
c
d
10
20
30
100
200
300
```

```
In [187... def mychain(*args):
              for one_arg in args:
                  for one_item in one_arg:
                      yield one_item

              for one_item in mychain(s, mylist, t):
                  print(one_item)
```

```
a
b
c
d
10
20
30
100
200
300
```

```
In [189... s = 'abcd'
mylist = [10, 20, 30]

list(zip(s, mylist))
```

```
Out[189... [('a', 10), ('b', 20), ('c', 30)]
```

```
In [190... import itertools

s = 'abcd'
mylist = [10, 20, 30]

list(itertools.zip_longest(s, mylist))
```

```
Out[190... [('a', 10), ('b', 20), ('c', 30), ('d', None)]
```

```
In [191... list(itertools.zip_longest(s, mylist, fillvalue='!!!!'))
```

```
Out[191... [('a', 10), ('b', 20), ('c', 30), ('d', '!!!!')]
```

Exercise: Config file

Given a text file of `key=value` lines, mixed with `#` comments and blank lines:

1. Build a dict from it, using a dict comprehension that iterates directly over the open file object.
2. Filter out the comments and blanks in the comprehension
3. Merge it onto a `DEFAULTS` dictionary, so that the file takes priority over the original defaults.

```
DEFAULTS = {  
    'host': '0.0.0.0',  
    'port': '80',  
    'timeout': '60',  
    'debug': 'true',  
    'loglevel': 'info',  
}
```

<https://practice.lernerpython.com/classroom/bfb596ff6a/ex-3>

The file you can read from is called dictconfig.txt.

```
In [196... DEFAULTS = {  
    'host': '0.0.0.0',  
    'port': '80',  
    'timeout': '60',  
    'debug': 'true',  
    'loglevel': 'info',  
}  
  
filename = 'dictconfig.txt'  
  
{  
    one_line.split('=')[0] : one_line.split('=')[1].strip()  
    for one_line in open(filename)  
    if not one_line.startswith('#')  
    if not one_line.startswith('\n')  
}
```

```
Out[196... {'host': 'localhost', 'port': '8080', 'timeout': '30', 'debug': 'false'}
```

```
In [193... !cat dictconfig.txt
```

```
# server settings  
host=localhost  
port=8080
```

```
timeout=30  
debug=false
```

```
In [198... DEFAULTS = {  
    'host': '0.0.0.0',  
    'port': '80',  
    'timeout': '60',  
    'debug': 'true',  
    'loglevel': 'info',  
}
```



```
filename = 'dictconfig.txt'

DEFAULTS = {
    one_line.split('=')[0] : one_line.split('=')[1].strip()
    for one_line in open(filename)
    if not one_line.startswith(('#', '\n'))
}
```

```
Out[198]: {'host': 'localhost',
           'port': '8080',
           'timeout': '30',
           'debug': 'false',
           'loglevel': 'info'}
```

```
In [202]: DEFAULTS = {
           'host': '0.0.0.0',
           'port': '80',
           'timeout': '60',
           'debug': 'true',
           'loglevel': 'info',
           }

filename = 'dictconfig.txt'

with open(filename) as f:
    d = DEFAULTS | {
        one_line.split('=')[0] : one_line.split('=')[1].strip()
        for one_line in f
        if not one_line.startswith(('#', '\n'))
    }
    # here, just before we exit the "with" block, __exit__ method runs, flushes
    print(d)
```

```
{'host': 'localhost', 'port': '8080', 'timeout': '30', 'debug': 'false', 'loglevel': 'info'}
```

The most controversial thing ever in Python: The walrus operator

For years, Python people were happy (smug?) to know that we couldn't mix assignment with conditions in `while` loops.

```
In [203]: # this never worked

while name = input('Enter your name: ').strip():
    print(f'Hello, {name}!')
```

Cell In[203], line 1

```
while name = input('Enter your name: ').strip():
    ^
```

SyntaxError: invalid syntax. Maybe you meant '==' or ':=' instead of '='?

```
In [205... while True:
    name = input('Enter your name: ').strip()

    if not name:
        break

    print(f'Hello, {name}!')
```

Hello, Reuven!
Hello, asdfsas!

```
In [207... # in Python 3.8, we added a new operator -- the walrus operator, aka :=
# officially, the walrus is known as the "assignment expression operator."

# if you want to assign and also get the value, you can use it here.

while name := input('Enter your name: ').strip():
    print(f'Hello, {name}!')
```

Hello, asdfsafd!
Hello, asdfdasfdsafasdfjasdl!

The good news about the walrus

You can only use it where you CANNOT use the regular assignment operator. (And vice versa.)

So you cannot say

```
In [208... x := 5
```

```
Cell In[208], line 1
    x := 5
      ^
SyntaxError: invalid syntax
```

```
In [217... {
    fields[0] : fields[1]
    for one_line in open('dictconfig.txt')
    if not one_line.startswith(('#', '\n'))
    if (fields := one_line.strip().split('='))
}
```

```
Out[217... {'host': 'localhost', 'port': '8080', 'timeout': '30', 'debug': 'false'}
```

```
In [218... fields
```

```
Out[218... ['debug', 'false']
```

```
In [219... x = 100

def myfunc():
    x += 1 # because we're assigning to a local variable, the compiler name
```

```
print(x)

print(x)
myfunc()
print(x)
```

100

UnboundLocalError

Traceback (most recent call last)

Cell In[219], line 8

```
4     x += 1
5     print(x)
6
7 print(x)
----> 8 myfunc()
9 print(x)
```

Cell In[219], line 4, in myfunc()

```
3 def myfunc():
----> 4     x += 1
5     print(x)
```

UnboundLocalError: cannot access local variable 'x' where it is not associated with a value

```
In [220...] def add_one(x=[]):
              x.append(1)
              return x

mylist = [10, 20, 30]
add_one(mylist)
```

Out[220...] [10, 20, 30, 1]

```
In [221...] mylist
```

Out[221...] [10, 20, 30, 1]

```
In [222...] # what if I call the function without any argument?

add_one()
```

Out[222...] [1]

```
In [223...] add_one()
```

Out[223...] [1, 1]

```
In [224...] add_one.__defaults__
```

Out[224...] ([1, 1],)

Next up

1. Type hints + `mypy`
2. dataclasses

Resume at :45

```
In [232... DEFAULTS = {
    'host': '0.0.0.0',
    'port': '80',
    'timeout': '60',
    'debugger': 'true',
    'loglevel': 'info'
}

filename = 'dictconfig.txt'

with open(filename) as f:
    d = {
        one_line.split('=')[0] : one_line.split('=')[1]
        for one_line in f
        if not one_line.startswith(('#', '\n'))
    }
```

```
In [234... one_line = open(filename).readline()
```

```
In [236... one_line.startswith('#')
```

```
Out[236... True
```

```
In [237... one_line.startswith('\n')
```

```
Out[237... False
```

```
In [238... # here, we're calling str.startswith with a tuple of strings that we're looking for
one_line.startswith(('#', '\n'))
```

```
Out[238... True
```

```
In [239... # here, we're calling str.startswith with two positional arguments
# if you pass multiple positional arguments, the first is the "startswith" string
# and the second is the index at which you want to start searching for a match
one_line.startswith('#', '\n')
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[239], line 1
----> 1 one_line.startswith('#', '\n')

TypeError: slice indices must be integers or None or have an __index__ method
```

```
In [240... help(str.startswith)
```

Help on method descriptor startswith:

startswith(self, prefix[, start[, end]], /) unbound builtins.str method
Return True if the string starts with the specified prefix, False otherwise.

prefix
A string or a tuple of strings to try.
start
Optional start position. Default: start of the string.
end
Optional stop position. Default: end of the string.

In [241... *# Python took our arguments and created this:*

```
one_line['\n':] == '#'
```

```
-----  
TypeError                                Traceback (most recent call last)  
Cell In[241], line 3  
      1 # Python took our arguments and created this:  
      2  
----> 3 one_line['\n':] == '#'  
  
TypeError: slice indices must be integers or None or have an __index__ method
```

Type hints

Python is a famously dynamically typed language. What does that mean?

1. Variables are not aliases to locations in memory. Variables in Python are references to objects -- every object is on the heap, and every reference just points to that location in memory.
2. This means that every variable is just an address in memory, which means that they're all the same size. So every variable, for every value, can be the same size, and we don't need to declare/pre-allocate.

Except... this means that someone can accidentally assign a value to a variable that is a different type than it had before.

In [242... `x = 5`
`type(x)`

Out[242... `int`

In [243... `x = 'abcde'`
`type(x)`

Out[243... `str`

Dynamically typed languages are famously more elegant and flexible.

But they are also prone to run-time errors, because we cannot check the types at compile time.

```
In [244... def myfunc():  
            asdfsafsafsafsafa
```

```
In [245... myfunc()
```

```
-----  
NameError                                Traceback (most recent call last)  
Cell In[245], line 1  
----> 1 myfunc()  
  
Cell In[244], line 2, in myfunc()  
      1 def myfunc():  
----> 2     asdfsafsafsafsafa  
  
NameError: name 'asdfsafsafsafsafa' is not defined
```

In the last decade or so, Python has added "type hints" or "type annotations" to the language.

```
In [246... def hello(name):  
            return f'Hello, {name}!'
```

```
In [247... hello('Reuven')
```

```
Out[247... 'Hello, Reuven!'
```

```
In [248... hello(5)
```

```
Out[248... 'Hello, 5!'
```

```
In [249... hello([10, 20, 30])
```

```
Out[249... 'Hello, [10, 20, 30]!'
```

```
In [250... hello(hello)
```

```
Out[250... 'Hello, <function hello at 0x10a821220>!'
```

```
In [251... # luckily, we can now add type annotations  
  
def hello(name : str) -> str:    # name should be a string, and the function  
            return f'Hello, {name}!'
```

```
In [252... hello('Reuven')
```

```
Out[252... 'Hello, Reuven!'
```

```
In [253... hello(5)
```

```
Out[253... 'Hello, 5!'
```

```
In [254... hello([10, 20, 30])
```

```
Out[254... 'Hello, [10, 20, 30]!'
```

```
In [255... hello(hello)
```

```
Out[255... 'Hello, <function hello at 0x10a821380>!'
```

Python will do NOTHING WHATSOEVER with the type annotations, other than put them into a special dict.

The idea is that your tools -- your IDE or external checkers -- will use these.

```
In [256... hello.__annotations__
```

```
Out[256... {'name': str, 'return': str}
```

Who should use type annotations?

Some people would say, "everyone."

I'd say:

- The more people on a project
- The more important the project
- The larger the project

Each of these should push you to use type annotations.

Also: If you're doing agentic coding with Claude Code, you MUST MUST MUST use type annotations and enforce them.

Classes

Everything in Python is an object. But the type system was a bit weird in Python 2:

- You had two ways to define classes, "new-style" (where you inherited from `object`) and "old-style" (where you didn't)
- `super` required that you pass arguments to it, to say what the instance and class were
- There were some "dunder" methods, but not a ton.

In modern Python (3):

- We only have one style of class, all new-style, and you don't have to inherit from `object`
- `super` knows (through trickery) what its instance and class are, so you can invoke it without any arguments
- There are *tons* of new dunder methods in classes

The classes that you write, and the built-in classes, work in the same way.

However, you still have to write a lot of code to get your class to work:

- `__init__` for initialization, and to set attributes
- `__str__` and `__repr__` to indicate how instances should turn themselves into strings

This boilerplate is kind of annoying!

For years, PyPI has offered `attrs`, a package that reduces this boilerplate.

Python now includes (as of 3.9?) something called `dataclasses`, which is a younger sibling to `attrs`.

```
In [263... from dataclasses import dataclass

@dataclass
class Person:
    first_name : str
    last_name  : str
    shoe_size  : int
```

You define a class, and decorate it with `@dataclass`. You then define one or more attributes at the *class* level, giving them type hints. Those hints force Python to define them as class attributes.

It automatically:

- Defines `__init__`
- Defines `__repr__`

```
In [264... p1 = Person()
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[264], line 1
----> 1 p1 = Person()

TypeError: Person.__init__() missing 3 required positional arguments: 'first_name', 'last_name', and 'shoe_size'
```


In [262... p1

Out[262... Person(first_name='(generic first_name)', last_name='(generic last_name)', shoe_size=20)

In [272... **from** dataclasses **import** dataclass

```
@dataclass
class Person:
    first_name : str
    last_name  : str
    shoe_size  : int
    hobbies    : list
```

In [267... p1 = Person('fn1', 'ln1', 1, ['Python', 'More Python'])

In [268... p2 = Person('fn1', 'ln1', 1, ['cooking', 'sleeping'])

In [269... p1

Out[269... Person(first_name='fn1', last_name='ln1', shoe_size=1, hobbies=['Python', 'More Python'])

In [270... p2

Out[270... Person(first_name='fn1', last_name='ln1', shoe_size=1, hobbies=['cooking', 'sleeping'])

In [273... *# let's have a default for hobbies!*

from dataclasses **import** dataclass

```
@dataclass
class Person:
    first_name : str
    last_name  : str
    shoe_size  : int
    hobbies    : list = []
```

```

ValueError                                Traceback (most recent call last)
Cell In[273], line 5
      1 # let's have a default for hobbies!
      2
      3 from dataclasses import dataclass
      4
----> 5 @dataclass
      6 class Person:
      7     first_name : str
      8     last_name : str

File ~/.local/share/uv/python/cpython-3.14.2-macos-aarch64-none/lib/python3.14/dataclasses.py:1442, in dataclass(cls, init, repr, eq, order, unsafe_hash, frozen, match_args, kw_only, slots, weakref_slot)
    1439     return wrap
    1441 # We're called as @dataclass without parens.
-> 1442 return wrap(cls)

File ~/.local/share/uv/python/cpython-3.14.2-macos-aarch64-none/lib/python3.14/dataclasses.py:1432, in dataclass.<locals>.wrap(cls)
    1431 def wrap(cls):
-> 1432     return _process_class(cls, init, repr, eq, order, unsafe_hash,
    1433                          frozen, match_args, kw_only, slots,
    1434                          weakref_slot)

File ~/.local/share/uv/python/cpython-3.14.2-macos-aarch64-none/lib/python3.14/dataclasses.py:1066, in _process_class(cls, init, repr, eq, order, unsafe_hash, frozen, match_args, kw_only, slots, weakref_slot)
    1063     kw_only = True
    1064     else:
    1065         # Otherwise it's a field of some type.
-> 1066     cls_fields.append(_get_field(cls, name, type, kw_only))
    1068 for f in cls_fields:
    1069     fields[f.name] = f

File ~/.local/share/uv/python/cpython-3.14.2-macos-aarch64-none/lib/python3.14/dataclasses.py:917, in _get_field(cls, a_name, a_type, default_kw_only)
    913 # For real fields, disallow mutable defaults. Use unhashable as a proxy
    914 # indicator for mutability. Read the __hash__ attribute from the class,
    915 # not the instance.
    916 if f._field_type is _FIELD and f.default.__class__.__hash__ is None:
--> 917     raise ValueError(f'mutable default {type(f.default)} for field {f.name} is not allowed: use default_factory')
    918
    920 return f

ValueError: mutable default <class 'list'> for field hobbies is not allowed: use default_factory

```

In [274... `from dataclasses import dataclass, field`

```

@dataclass
class Person:

```

```
first_name : str
last_name  : str
shoe_size  : int
hobbies    : list = field(default_factory = list)
```

```
In [275... p = Person('fn', 'ln', 20)
```

```
In [276... p.hobbies
```

```
Out[276... []
```

```
In [ ]:
```